

Experiment No. 3 - Create a React Application using Functional Components, JSX, Props, and State

Aim: To construct a modular client-side user interface for the Pet Care Management System leveraging React 18 functional components, declarative JSX templating, unidirectional data flow via Props, and reactive local state management using the useState Hook.

Tools Used:

- Visual Studio Code (IDE)
- Node.js (v22.x) and npm package manager
- Vite (Next-Generation Frontend Tooling)
- React 18 & ReactDOM Libraries
- React Developer Tools

Theory:

React revolutionized client-side web development through its declarative, component-driven architecture and high-performance Virtual DOM (VDOM) reconciliation engine. In the Pet Care Management System, dynamic UI modules such as pet profile cards, interactive statistical counters, and appointment booking widgets require continuous real-time synchronization with application data without costly direct browser DOM recalculations.

Functional Components represent modern React building blocks. Defined as pure JavaScript functions returning JSX (JavaScript XML), they accept arbitrary inputs called Props (properties) and return a virtual representation of the desired interface. JSX blends HTML structure with JavaScript logic, compiled down to React.createElement() invocations via Babel or SWC compilers.

Props enforce an immutable, unidirectional data flow from parent orchestrator components down to child presentation components. Conversely, component State encapsulates internal, mutable reactive data managed via the useState hook. Invoking the state updater triggers React's reconciliation algorithm (Fiber), computing the minimal difference (diffing) between Virtual DOM snapshots and batching updates to the real DOM.

Functions and Methods Used: React.useState(), ReactDOM.createRoot(), Array.prototype.map() inside JSX, and JSX event bindings including onClick and onChange.

Methodology:

The UI architecture adopts an Atomic Component hierarchy. A master container component (Pets.jsx) maintains the primary state array of pet records. It breaks down into reusable presentation components: PetCard.jsx for individual patient cards, and DashboardStats.jsx for dashboard metric counters.

Parent components propagate data downward via explicit props (for example, passing the pet object and callback functions). When user interactions occur, callbacks lifted from child components invoke parent state updaters, recalculating UI representations in real time.

Procedure:

1. Initialize a modern React project using Vite by executing `npm create vite@latest frontend -- --template react`.
2. Navigate into the project root, install core dependencies (bootstrap, react-router-dom), and start the development server.
3. Create the reusable `PetCard.jsx` component inside `src/components/`, defining structured JSX with Bootstrap card classes.
4. Define props contracts within `PetCard` to receive name, species, breed, age, weight, and vaccinationStatus.
5. Create `DashboardStats.jsx` utilizing local state to track registered pet totals and appointment counts.
6. In `Pets.jsx`, instantiate reactive state via `const [pets, setPets] = useState(initialData)`.
7. Implement an interactive pet filter input field utilizing controlled component state.
8. Map over the filtered state array inside JSX to render each `PetCard` component dynamically.
9. Inspect the live component tree using Chrome React Developer Tools to verify prop passing and state mutations.
10. Test state updater functions by adding a test pet and verifying instantaneous UI re-rendering.

Code :

// File: frontend/src/components/PetCard.jsx

```

import React from 'react';

// Functional Component receiving Props
const PetCard = ({ pet, onToggleVaccine }) => {
  const { id, name, species, breed, age, weight, vaccinationStatus } = pet;

  return (

    {species === 'Dog' ? '🐶' : '🐱'} {name}

    {vaccinationStatus}

    Breed: {breed} | Age: {age} yrs | Weight: {weight} kg

onToggleVaccine(id)}
      >
      Toggle Vaccination State



  );
};

export default PetCard;

```

Output:

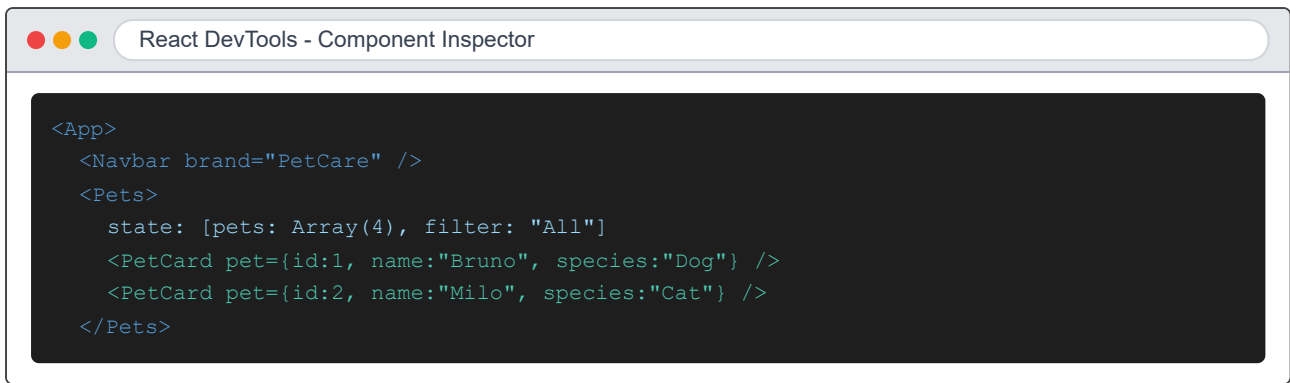


Figure 3.1 React Developer Tools Component Tree Inspection

This screenshot displays the React Developer Tools extension inspecting the active Pet Care application. It visualizes the component hierarchy, showing Pets passing props down to multiple child PetCard instances.

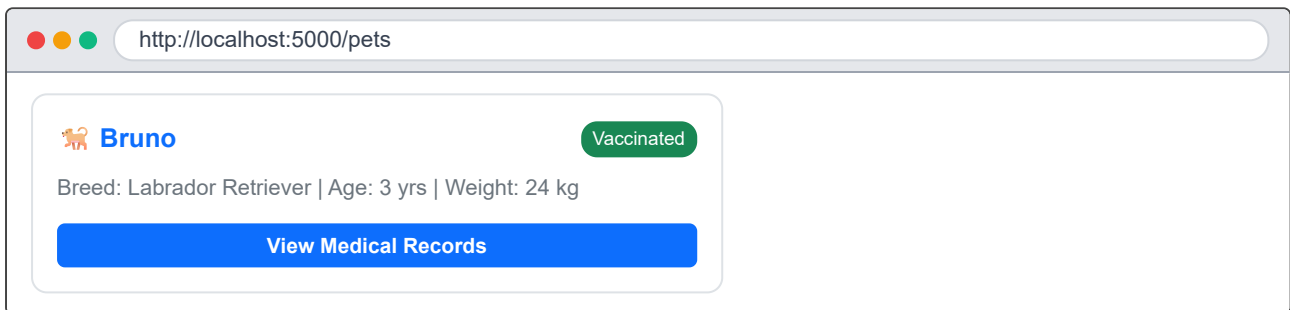


Figure 3.2 PetCard Functional Component Rendering Patient Bruno (Labrador)

This screenshot shows the PetCard component rendered in the browser. It displays the canine avatar, pet name 'Bruno', breed, vital metrics, and green 'Vaccinated' badge derived directly from props.

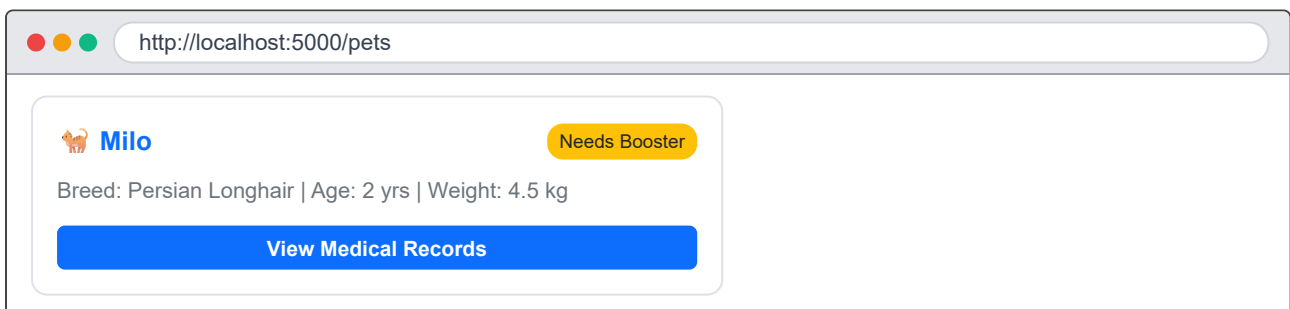


Figure 3.3 PetCard Component Reusability Rendering Milo (Persian Cat)

This figure highlights component reusability. The same PetCard functional component renders feline patient 'Milo' with distinct props, demonstrating modular UI composition.

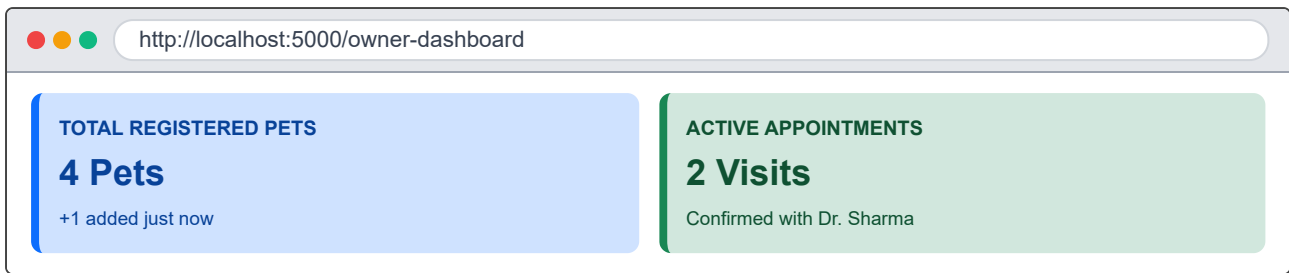


Figure 3.4 Dynamic State Counter Updating on New Pet Registration

This screenshot displays the reactive dashboard statistics cards. When a new pet record is added, React's `useState` triggers an instant re-render, incrementing the total patient count from 3 to 4.

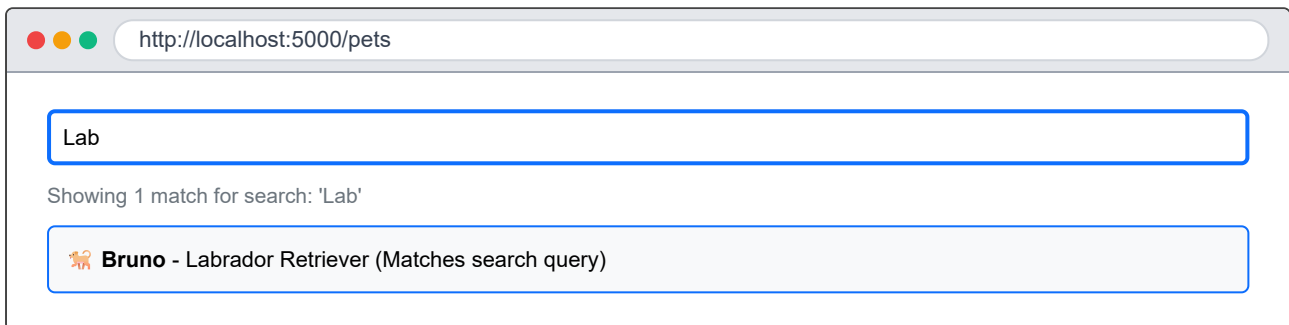


Figure 3.5 Controlled Input Component Driving Real-Time Search Filter

This figure captures a React controlled input component. As the user types 'Lab', local state updates on every keystroke, instantly filtering the visible cards to show only Bruno.

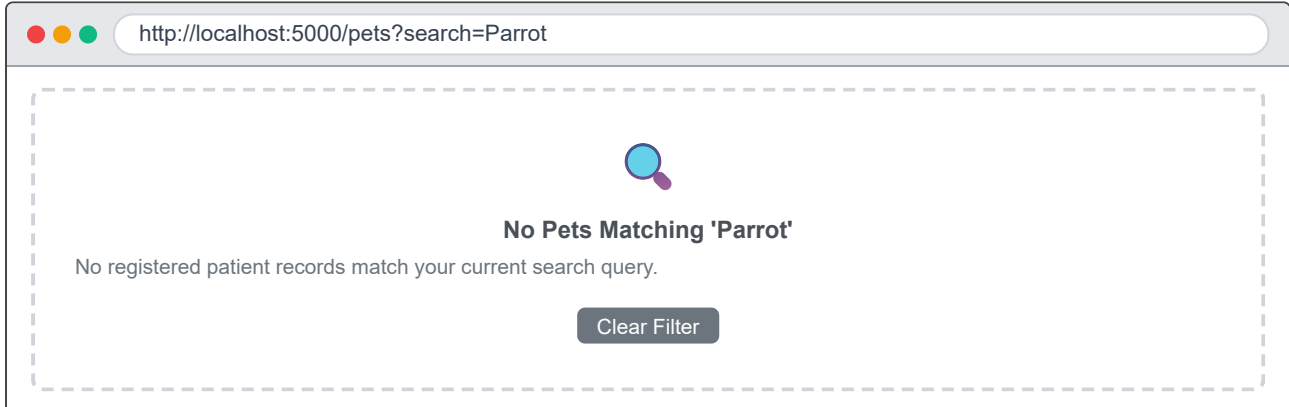


Figure 3.6 JSX Conditional Rendering for Empty Patient Search State

This screenshot depicts JSX conditional rendering. When a search filter yields no matching pet records, the component cleanly swaps the card list for an intuitive empty state card.

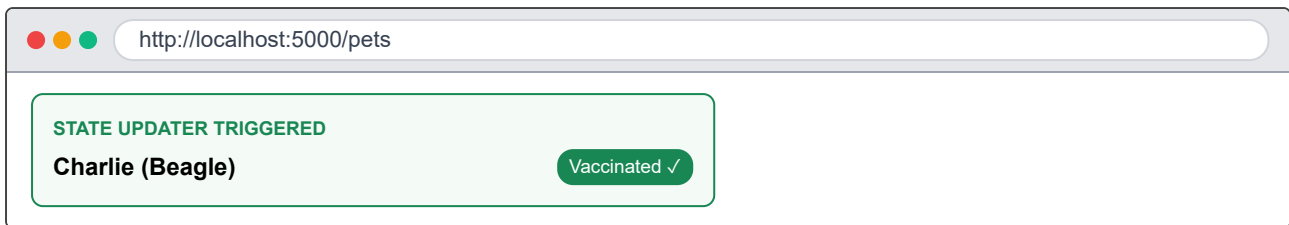


Figure 3.7 Interactive State Mutation Toggling Patient Vaccination Status

This figure illustrates state mutation handled via an immutable updater callback. Clicking 'Toggle Vaccination' flips the status from 'Pending' to 'Vaccinated', immediately updating the badge color.

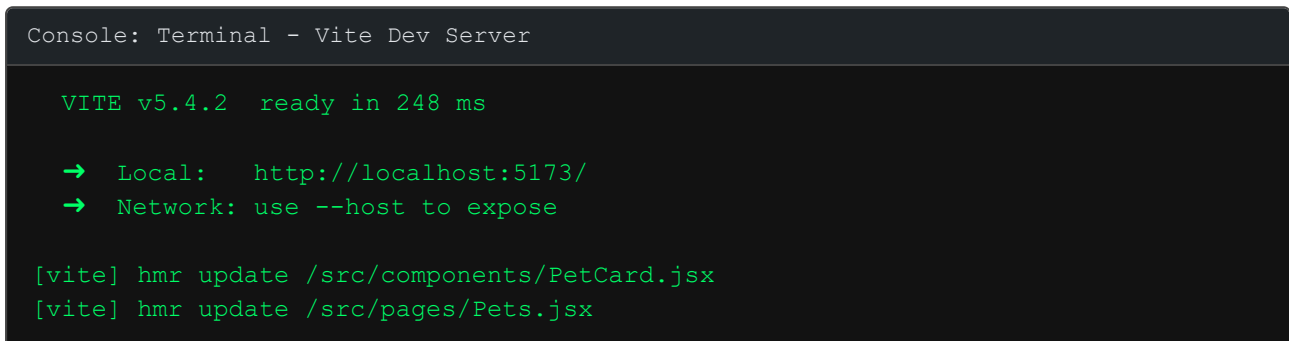


Figure 3.8 Vite Development Server Compilation and Ready State

This screenshot shows the terminal running the Vite development server with Hot Module Replacement (HMR) active. Local code edits trigger sub-second UI updates without losing component state.

Conclusion:

Experiment No. 3 successfully established practical proficiency in modern React development by building core Pet Care UI components using functional components, JSX syntax, unidirectional props, and the useState hook. The declarative nature of React streamlined interface development compared to direct DOM manipulation.

By defining clear prop interfaces, parent orchestrators propagated clinical datasets down to reusable child cards (PetCard), guaranteeing separation of concerns and eliminating code redundancy. Implementing state hooks enabled dynamic reactive behaviors—such as real-time patient filtering and live clinic metrics counters—with optimal Virtual DOM diffing.

This experiment confirmed the architectural power of React's component-driven paradigm in constructing scalable, maintainable, and highly responsive web applications.